Entregable 04 · ADR (Architecture Decision Records)

Siete decisiones de arquitectura, cada una como archivo .md en /docs/adr/ del repositorio.

Campo	Valor común a todos los ADR
Formato	Estado · Contexto · Decisión · Alternativas · Consecuencias · Trazabilidad
Ubicación	Un archivo .md por ADR en /docs/adr/
Fecha	Octubre de 2026
Estado	Aceptado

ADR-001 · Usar Domain Model con Service Layer como patrón de lógica de negocio

Sección	Detalle
Estado	Aceptado (octubre 2026)
Contexto	El BRD tiene 12 reglas implícitas, 4 necesidades de prioridad ALTA que son invariantes (N1, N2, N3, N5) y el problema actual de criterios inconsistentes en devoluciones.
Decisión	El núcleo se modela como Domain Model (Aggregates, Value Objects, Domain Services, Domain Events) expuesto únicamente por Application Services. Las consultas de solo lectura (US01, US15, US17) pueden saltarse el dominio.
Alternativas	Transaction Script (rápido, pero duplica reglas); Table Module y Active Record (acoplan reglas a la estructura de tablas).
Consecuencias	(+) Reglas en un solo lugar y probables con Gherkin. (+) Cambios de política localizados. (−) Mayor curva de aprendizaje y más clases iniciales.
Trazabilidad	N1, N2, N3, N5; US02–US09; Sección 1 de este documento.

ADR-002 · NumeroDeSerie como Aggregate propio con unicidad global

Sección	Detalle

Sección	Detalle
Estado	Aceptado (octubre 2026)
Contexto	Los seriales son únicos a nivel sistema (BRD, supuestos), obligatorios en computadoras, celulares y tablets, y hoy son difíciles de rastrear (N1, N6).
Decisión	NumeroDeSerie es un Aggregate Root independiente (no una lista dentro de Producto ni de StockSucursal), con identidad = su valor, estados Disponible/Vendido/Devuelto/Dañado e historial de EventoSerial. La unicidad se garantiza con restricción UNIQUE en la base de datos además de la validación en el dominio.
Alternativas	Guardar seriales como atributo de LineaVenta (no permite historial ni validar duplicados entre ventas); lista dentro de StockSucursal (aggregate enorme y bloqueos innecesarios).
Consecuencias	(+) Historial completo para US11. (+) Cero seriales repetidos. (−) Cada venta serializada toca dos aggregates, resuelto dentro de una transacción del Application Service.
Trazabilidad	N1, N6; US03, US11; reglas 2 y 9.

ADR-003 · Reserva de stock y bloqueo de doble venta

Sección	Detalle
Estado	Aceptado (octubre 2026)
Contexto	Hoy se venden por error productos apartados (N2, ALTA); la meta es cero dobles ventas en 30 días. Hay 5 sucursales y un PdV por sucursal.
Decisión	El apartado incrementa StockSucursal.reservado dentro de la misma transacción que crea el Apartado. disponible() = total − reservado − dañado − en tránsito es la única forma de calcular lo vendible. La concurrencia se controla con bloqueo optimista (columna de versión en StockSucursal): si dos ventas compiten por la última unidad, una falla y reintenta.

Sección	Detalle
Alternativas	Bloqueo pesimista de filas (más lento con PdV remotos); validar solo en la interfaz del PdV (no confiable).
Consecuencias	(+) El sistema bloquea la venta de lo reservado (regla 4). (+) Una sola fórmula en todo el sistema. (−) Requiere manejo de reintentos en los Application Services.
Trazabilidad	N2; US04, US05; reglas 1 y 4. Nota: unifica la fórmula de disponibilidad, ver sección 6.

ADR-004 · Cancelación de apartados vencidos mediante proceso batch de fin de día

Sección	Detalle
Estado	Aceptado (octubre 2026)
Contexto	El apartado dura 14 días fijos y vence automáticamente (N4, regla 12). El BPMN modela la rama "Fin de día/Auditoría".
Decisión	Un job diario invoca ApartadoAppService.CancelarApartadosVencidos(fechaCorte). El método es idempotente (solo procesa apartados en estado Activo con fechaVencimiento < hoy), libera el stock, notifica al cliente y registra auditoría; si falla a la mitad, se puede volver a ejecutar sin duplicar efectos.
Alternativas	Cancelar al consultar el apartado (deja productos bloqueados si nadie lo consulta); temporizador por apartado (complejo y frágil).
Consecuencias	(+) Sencillo y auditable. (−) Un apartado vencido puede seguir bloqueado hasta el cierre del día, aceptado por el negocio.
Trazabilidad	N4; US07; regla 12; rama Fin de día del BPMN.

ADR-005 · Reglas de devolución en un Domain Service con aprobación de Supervisor

Sección	Detalle
Estado	Aceptado (octubre 2026)

Sección	Detalle
Contexto	Los criterios de devolución se aplican hoy de forma inconsistente (N5). Reglas: 30 días, comprobante O aprobación de Supervisor, producto sin daño, aprobación si monto > $500, reembolso por el método original.
Decisión	PoliticaDevolucionService centraliza la evaluación (patrón Specification para cada criterio); Devolucion modela el ciclo de vida y guarda AprobacionSupervisor. Ningún Application Service ni pantalla del PdV re-implementa estas reglas.
Alternativas	Validar en el PdV (inconsistente entre sucursales); reglas en la base de datos (difíciles de probar y de auditar).
Consecuencias	(+) Criterio único en las 5 sucursales. (+) Umbrales fáciles de cambiar en un solo lugar. (−) El Supervisor es remoto (restricción BRD), por lo que la autorización debe ser una operación asíncrona o remota.
Trazabilidad	N5; US08, US09; reglas 5, 6, 7, 8.

ADR-006 · Auditoría mediante Domain Events y bitácora append-only

Sección	Detalle
Estado	Aceptado (octubre 2026)
Contexto	La regla 9 exige registrar TODO con timestamp, actor y justificación; N3 y N6 requieren reconstruir qué pasó con inventario y seriales.
Decisión	Los Aggregates emiten Domain Events; AuditoriaEventHandler los guarda como RegistroAuditoria inmutable en la misma transacción del caso de uso. No se permite UPDATE ni DELETE sobre la bitácora.
Alternativas	Triggers en base de datos (pierden el actor y la justificación de negocio); logs de aplicación en texto (no consultables).
Consecuencias	(+) Trazabilidad total, base para US11 y para

Sección	Detalle
	conciliar diferencias. (−) Crece la tabla de auditoría, se resuelve con archivado periódico.
Trazabilidad	N3, N6; US07, US10, US11; regla 9.

ADR-007 · Arquitectura en capas e integración con el PdV Windows existente

Sección	Detalle
Estado	Aceptado (octubre 2026)
Contexto	Restricción tecnológica del BRD: el PdV Windows actual debe integrarse; hay 5 sucursales y 1 Supervisor remoto; el comprobante debe cumplir normativa fiscal.
Decisión	Arquitectura en capas (Presentación/PdV → Application → Domain → Infrastructure). El PdV se conecta solo mediante AdaptadorPdV, que invoca los Application Services; el dominio no depende de la UI ni de la base de datos (repositorios como interfaces). Stack propuesto (supuesto, a confirmar con el docente): C#/.NET por compatibilidad con Windows.
Alternativas	Que el PdV acceda directo a la base de datos (rompe todas las reglas); reescribir el PdV (fuera del alcance del BRD).
Consecuencias	(+) El PdV actual se reemplaza o evoluciona sin tocar el dominio. (+) Dominio probado sin infraestructura. (−) Hay que construir el adaptador y su mapeo de datos.
Trazabilidad	Restricciones del BRD; N7, N8; US12–US14.